

Assignment 01: Project TrafficGuard

Course: Agentic AI Systems – From Concepts to Practice
Instructor: Prof. Karthik Vaidhyanathan
TAs: Aviral Gupta, Aneetta Sara Shany, Ch Pavan Harshit, Shreyash Chandak

Topic: Python Fundamentals for AI Engineering
Prerequisites: None (Zero to Hero)

Mission Briefing

Welcome, Lead Engineer.

The City Council has commissioned the "Future City" initiative. Before we can build advanced Artificial Intelligence agents, we must first build the basic digital infrastructure to process data.

Your task is to write the code for the **TrafficGuard Legacy System**. This system will manually process vehicle data from roadside sensors using explicit logic (rules). This infrastructure will serve as the foundation for the AI systems we will build in Phase II.

Learning Objectives

By completing this assignment, you will master:

- **The Environment:** Navigating Jupyter Notebooks (Cells, Markdown, Execution).
- **Data Representation:** Using Variables (Strings, Integers, Floats).
- **Logic Control:** Writing explicit rules using `if`, `elif`, and `else`.
- **Data Structures:** Managing traffic queues using Lists and Dictionaries.
- **Automation:** Creating Functions and Loops to process data streams.
- **Data Viz:** "Seeing" the data before coding.
- **Linear Regression:** Predicting values with a simple line.
- **The Neuron:** Understanding the math of classification ($w \cdot x + b$).
- **AI Safety:** Building guardrails for when the AI hallucinates.

Assignment Tasks

Phase I: Building the Digital Infrastructure

Part 0: The Command Center (Jupyter Basics)

Objective: Overcome the fear of the coding interface.

You have just logged into the TrafficGuard central terminal (your Jupyter Notebook).

1. Create a Markdown Cell at the top of your notebook. Add a Title header (#) and your Name/Student ID.
2. Create a Code Cell that prints a system boot message:

```
print("System Booting... TrafficGuard v1.0 is live.")
```

The Safety Drill: Create an infinite loop (code provided in the notebook) and practice using the "Interrupt Kernel" (Stop Button) to regain control.

Part 1: The Sensors (Variables & Data Types)

Objective: Understanding how to store sensor readings.

Our roadside sensors detect a vehicle. You must store this data in memory.

1. Define three variables to represent a single detected car:
   - `plate_id` (String): e.g., `"KA-05-MJ-1029"`
   - `vehicle_length` (Float): The length in meters, e.g., `4.5`
   - `is_emergency` (Boolean): `True` or `False`
2. Calculate the "Road Footprint" of the vehicle using the formula: area = vehicle_length $\times$ 2.5

(assuming standard width). Print the result.

Part 2: The Gatekeeper (Control Flow & Logic)

Objective: Writing the "Rules of the Road" using Logic.

In this phase, we are not using AI. We are using "Symbolic AI" (Hard-coded rules). Write a script that checks the variables from Part 1:

1. **The Heavy Duty Rule:** If `vehicle_length` is greater than `10` meters, print `"Class: Heavy Vehicle. Toll: $50"`.
2. **The Standard Rule:** If `vehicle_length` is less than or equal to `10`, print `"Class: Light Vehicle. Toll: $10"`.
3. **The Ambulance Exception:** Modify your logic. If `is_emergency` is `True`, the toll should be `$0`, regardless of length.

Part 3: The Logbook (Lists & Dictionaries)

Objective: Handling multiple vehicles.

Single variables are not enough for a highway. We need data structures.

1. **The Queue (List):** Create a list called `traffic_queue` containing the lengths of the next 5 cars: `[4.5, 12.0, 3.2, 18.5, 5.0]`.
2. **Accessing Data:** Write code to print the length of the last vehicle in the queue.
3. **The Record (Dictionary):** Create a dictionary representing a specific truck with keys for `id`, `company`, and `weight`.

Part 4: Automation (Functions & Loops)

Objective: Processing data at scale.

We cannot write `if/else` statements manually for every car. We need a "Function"—a reusable machine.

1. Define a function named `calculate_toll(length, is_emergency)`.
2. Move your logic from Part 2 inside this function so it returns the price.
3. **Batch Processing:** Write a `for` loop that goes through your `traffic_queue` list. Inside the loop, call your function for every length and print the toll.

Part 5: The Debugging Challenge

Objective: Learning to read Error Messages.

A junior intern attempted to update the system but broke the code. The notebook contains a cell with specific errors (`SyntaxError` and `IndentationError`).

1. Run the broken cell to see the error message.
2. Fix the code so it executes without errors.

Reflection (The Bridge to AI)

Critical Thinking

In a Markdown cell at the end of your notebook, answer the following:

"In Part 2, you hard-coded that 10 meters is the cutoff for a heavy vehicle. What happens if a new vehicle type appears that is 9.9 meters long but weighs 50 tons? Your current code would classify it as 'Light'. How is this limitation related to the need for Machine Learning?"

Phase II: The Intelligent Upgrade

Objective: Upgrade the system from one-feature rules to data-driven classification.

CRITICAL INCIDENT REPORT

Status: SYSTEM FAILURE

Yesterday, a strong wind blew a large cardboard billboard onto Highway 4. The object was **15 meters long** but weighed only **0.2 tons**.

Your "Legacy System" used the rule: `if vehicle_length > 10: class = Truck`. It misclassified the billboard as a **Heavy Truck** and charged a `$50` toll. The City Council is unhappy.

New Mandate: Retire the hard-coded rules. Deploy a **Machine Learning** prototype that looks at patterns (**Weight and Length**) to make smarter decisions.

Part 6: Visualization (Seeing the Data)

Objective: Understanding the shape of the problem.

Before we build a model, we must verify our data assumptions.

1. **Generate Data:** Run the provided code block to generate a synthetic dataset of 100 vehicles.
2. **Visualize:** Use the provided `matplotlib` code to plot:
   - X-axis: Vehicle Length – Y-axis: Vehicle Weight
   - Color: Red for Trucks, Blue for Cars.
3. **Exploratory Analysis:** Look at the plot and answer in a markdown cell:
   - Is there a clean gap between the Blue cluster and Red cluster?
   - Imagine a vehicle with `length = 15`, `weight = 0.2` (The Billboard). Where would it appear on this graph? Does it sit inside the "Truck" cluster?

Part 7: The Line of Best Fit (Manual Regression)

Objective: Intuition for "Weights" and "Bias" using simple pricing.

Before we classify types, let's try to predict the toll price based on weight.

1. We assume the relationship is linear: Price $= (m \cdot \text{Weight}) + c$.
2. **Visualize & Minimize:** Run the helper function `visualize_regressor(m, c)`.
   - The tool visualizes the line and calculates the Average Error (Mean Squared Error).
3. **Manual Tuning:**
   - Start with `m = 0`, `c = 0`. Note the high error score.

- Adjust `m` (slope) and `c` (intercept) until the line passes through the data and the Error Metric is minimized (closest to 0).

4. **Concept Check:** What does `m` represent in the real world? (Hint: Cost per ton).

Part 8: The Manual Neuron (The MCP Model)

Objective: Implementing the math of a single neuron.

Now, let's switch to classification (Heavy vs Light). A neuron is just a line that separates two groups.

1. **Implement the Neuron Formula** as a Python function:

$$z = (w_1 \cdot \text{length}) + (w_2 \cdot \text{weight}) + \text{bias}$$

2. **The Activation:** If `z > 0`, return `1` (Heavy). If `z <= 0`, return `0` (Light).
3. **Interactive Tuning:** Use the provided function `visualize_decision_boundary(w1, w2, bias)`.
4. **Task:** Manually change the weights until the Decision Boundary (the line) perfectly separates the Red dots from the Blue dots.

Part 9: Automated Learning (Scikit-Learn)

Objective: Letting the machine find the weights.

Hand-tuning is tedious. We will use `scikit-learn` to calculate the optimal `w` and `b` instantly.

1. Import `LogisticRegression` and fit it to your training data.
2. **Predict:** Ask the model to classify a Delivery Van (`length = 6`, `weight = 3.5`).
3. **Compare:** Inspect `model.coef_` (weights) and `model.intercept_` (bias) with the manual values you found in Part 8. Are they close?

Part 10: Systems Engineering (Safety & Reliability)

Objective: Moving from "Model Building" to "AI Engineering".

Mathematical models lack common sense. We must wrap them in safety code.

1. **The Ghost Vehicle Test:** Create a vehicle with impossible physics:
   - `ghost_vehicle = [[4, -50]]` (`length = 4`, `weight = -50`)
2. Feed this to your AI model. It will likely predict "Car" or "Truck" confidently. Why is this dangerous?
3. **The Fix (Guardrails):** Implement a function `traffic_system_verified(length, weight)`:
   - **Layer 1:** Check standard logic (e.g., `if weight < 0: return "SENSOR ERROR"`).
   - **Layer 2:** If Layer 1 passes, send data to the AI model for prediction.

Reflection & Analysis

Final Report

The AI Engineer's Dilemma

"In Part 8, you manually found the weights. In Part 9, the computer found them. In Part 10, you had to write manual if statements again."

Question: If AI is so smart, why do we still need manual guardrails (Part 10)? Why can't we trust the math 100 percent?

Submission Guidelines

- Submit `TrafficGuard_YourName.ipynb`.
- Ensure your "Safety Guardrail" correctly catches the Ghost Vehicle.
- Your Reflection answer accounts for 30% of the grade.

Remember: A model is just math. An Agent is a safe, working system.